

18

CNNs for Financial Time Series and Satellite Images

In this chapter, we introduce the first of several specialized deep learning architectures that we will cover in *Part 4*. Deep **convolutional neural networks** (CNNs) have enabled superhuman performance in various computer vision tasks such as classifying images and video and detecting and recognizing objects in images. CNNs can also extract signals from time-series data that shares certain characteristics with image data and have been successfully applied to speech recognition (Abdel-Hamid et al. 2014). Moreover, they have been shown to deliver state-of-the-art performance on time-series classification across various domains (Ismail Fawaz et al. 2019).

CNNs are named after a linear algebra operation called a **convolution** that replaces the general matrix multiplication typical of feedforward networks (discussed in the last chapter) in at least one of their layers. We will show how convolutions work and why they are particularly well suited to data with a certain regular structure typically found in images but also present in time series.

Research into **CNN architectures** has proceeded very rapidly, and new architectures that improve benchmark performance continue to emerge. We will describe a set of building blocks consistently used by successful applications. We will also demonstrate how **transfer learning** can speed up learning by using pretrained weights for CNN layers closer to the input while fine-tuning the final layers to a specific task. We will also illustrate how to use CNNs for the specific computer vision task of **object detection**.

CNNs can help build a **trading strategy** by generating signals from images or (multiple) time-series data:

- **Satellite data** may signal future commodity trends, including the supply of certain crops or raw materials via aerial images of agricultural areas, mines, or transport networks like oil tankers. **Surveillance camera** footage, for example, from shopping malls, could be used to track and predict consumer activity.

- **Time-series data** encompasses a very broad range of data sources and CNNs have been shown to deliver high-quality classification results by exploiting their structural similarity with images.

We will create a trading strategy based on predictions of a CNN that uses time-series data that's been deliberately formatted like images and demonstrate how to build a CNN to classify satellite images.

More specifically, in this chapter, you will learn about the following:

- How CNNs employ several building blocks to efficiently model grid-like data
- Training, tuning, and regularizing CNNs for images and time-series data using TensorFlow
- Using transfer learning to streamline CNNs, even with less data
- Designing a trading strategy using return predictions by a CNN trained on time-series data formatted like images
- How to classify satellite images



You can find the code samples for this chapter and links to additional resources in the corresponding directory of the GitHub repository. The notebooks include color versions of the images.

How CNNs learn to model grid-like data

CNNs are conceptually similar to feedforward **neural networks** (NNs): they consist of units with parameters called weights and biases, and the training process adjusts these parameters to optimize the network's output for a given input according to a loss function. They are most commonly used for classification. Each unit uses its parameters to apply a linear operation to the input data or activations received from other units, typically followed by a nonlinear transformation.

The overall network models a **differentiable function** that maps raw data, such as image pixels, to class probabilities using an output activation function like softmax. CNNs use an objective function such as cross-entropy loss to measure the quality of the output with a single metric. They also rely on the gradients of the loss with respect to the network parameter to learn via backpropagation.

Feedforward NNs with fully connected layers do not scale well to high-dimensional image data with a large number of pixel values. Even the low-resolution images included in the CIFAR-10 dataset that we'll use in the next section contain 32×32 pixels with up to 256 different color values represented by 8 bits each. With three channels, for example, for the red, green, and blue channels of the RGB color model, a single unit in a fully connected input layer implies $32 \times 32 \times 3 = 3,072$ weights. A more standard resolution of 640×480 pixels already yields closer to 1 million weights for a single input unit. Deep architectures with several layers of meaningful width quickly lead to an exploding number of parameters that make overfitting during training all but certain.

A fully connected feedforward NN makes no assumptions about the local structure of the input data so that arbitrarily reordering the features has no impact on the training result. By contrast, CNNs make the **key assumption** that the **data has a grid-like topology** and that the **local structure matters**. In other words, they encode the assumption that the input has a structure typically found in image data: pixels form a two-dimensional grid, possibly with several channels to represent the components of the color signal. Furthermore, the values of nearby pixels are likely more relevant to detect key features such as edges and corners than faraway data points. Naturally, initial CNN applications such as handwriting recognition focused on image data.

Over time, however, researchers recognized **similar characteristics in time-series data**, broadening the scope for the productive use of CNNs. Time-series data consists of measurements at regular intervals that create a one-dimensional grid along the time axis, such as the lagged returns for a given ticker. There can also be a second dimension with additional features for this ticker and the same time periods. Finally, we could represent additional tickers using the third dimension.

A common CNN use case beyond images includes audio data, either in a one-dimensional waveform in the time domain or, after a Fourier transform, as a two-dimensional spectrum in the frequency domain. CNNs also play a key role in AlphaGo, the first algorithm to win a game of Go against humans, where they evaluated different positions on the grid-like board.

The most important element to encode the **assumption of a grid-like topology** is the **convolution** operation that gives CNNs their name, combined with **pooling**. We will see that the specific assumptions about the functional relationship between input and output data imply that CNNs need far fewer parameters and compute more efficiently.

In this section, we will explain how convolution and pooling layers learn filters that extract local features and why these operations are particularly suitable for data with the structure just described. State-of-the-art CNNs combine many of these basic building blocks to achieve the layered representation learning described in the previous chapter. We conclude by describing key architectural innovations over the last decade that saw enormous performance improvements.

From hand-coding to learning filters from data

For image data, this local structure has traditionally motivated the development of hand-coded filters that extract such patterns for the use as features in **machine learning (ML)** models.

Figure 18.1 displays the effect of simple filters designed to detect certain edges. The notebook `filter_example.ipynb` illustrates how to use hand-coded filters in a convolutional network and visualizes the resulting transformation of the image. The filters are simple $[-1, 1]$ patterns arranged in a 2×2 matrix, shown in the upper right of the figure. Below each filter, its effects are shown; they are a bit subtle and will be easier to spot in the accompanying notebook.

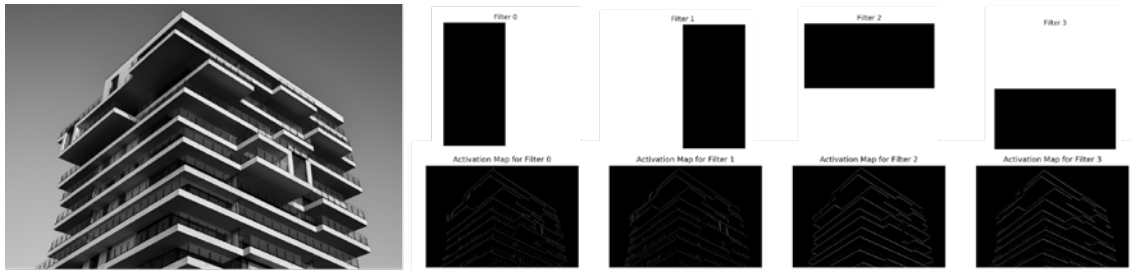


Figure 18.1: The result of basic edge filters applied to an image

Convolutional layers, by contrast, are designed to learn such local feature representations from the data. A key insight is to restrict their input, called the **receptive field**, to a small area of the input so it captures basic pixel constellations that reflect common patterns like edges or corners. Such patterns may occur anywhere in an image, though, so CNNs also need to recognize similar patterns in different locations and possibly with small variations.

Subsequent layers then learn to synthesize these local features to detect **higher-order features**. The linked resources on GitHub include examples of how to visualize the filters learned by a deep CNN using some of the deep architectures that we present in the next section on reference architectures.

How the elements of a convolutional layer operate

Convolutional layers integrate **three architectural ideas** that enable the learning of feature representations that are to some degree invariant to shifts, changes in scale, and distortion:

- Sparse rather than dense connectivity
- Weight sharing
- Spatial or temporal downsampling

Moreover, convolutional layers allow for inputs of variable size. We will walk through a typical convolutional layer and describe each of these ideas in turn.

Figure 18.2 outlines the set of operations that typically takes place in a three-dimensional convolutional layer, assuming image data is input with the three dimensions of height, width, and depth, or the number of channels. The range of pixel values depends on the bit representation, for example, $[0, 255]$ for 8 bits. Alternatively, the width axis could represent time, the height different features, and the channels could capture observations on distinct objects such as tickers.

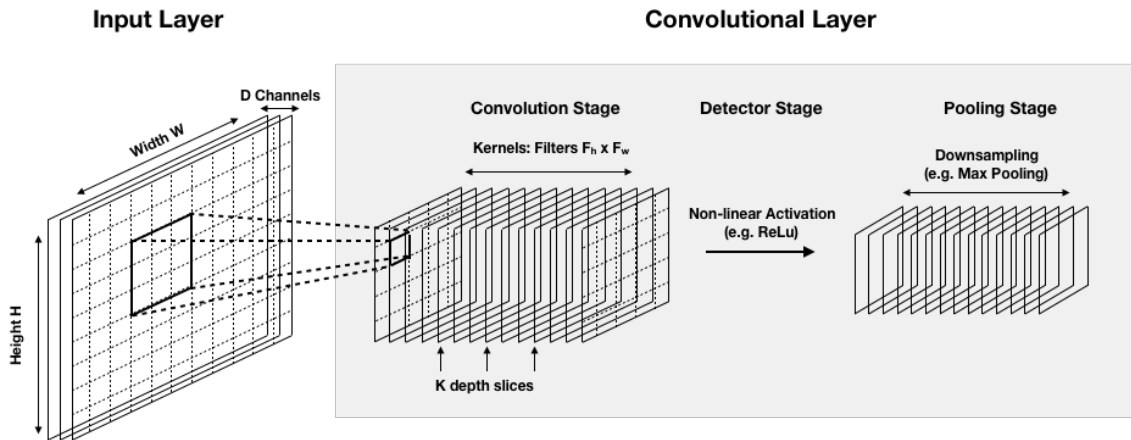


Figure 18.2: Typical operations in a two-dimensional convolutional layer

Successive computations process the input through the convolutional, detector, and pooling stages that we describe in the next three sections. In the example depicted in *Figure 18.2*, the convolutional layer receives three-dimensional input and produces an output of the same dimensionality.

State-of-the-art CNNs are composed of several such layers of varying sizes that are either stacked on top of each other or operate in parallel on different branches. With each layer, the network can detect higher-level, more abstract features.

The convolution stage – extracting local features

The first stage applies a filter, also called the **kernel**, to overlapping patches of the input image. The filter is a matrix of a much smaller size than the input so that its receptive field is limited to a few contiguous values such as pixels or time-series values. As a result, it focuses on local patterns and dramatically reduces the number of parameters and computations relative to a fully connected layer.

A complete convolutional layer has several **feature maps** organized as depth slices (depicted in *Figure 18.2*) so that each layer can extract multiple features.

From filters to feature maps

While scanning the input, the kernel is convolved with each input segment covered by its receptive field. The convolution operation is simply the dot product between the filter weights and the values of the matching input area after both have been reshaped to vectors. Each convolution thus produces a single number, and the entire scan yields a feature map. Since the dot product is maximized for identical vectors, the feature map indicates the degree of activation for each input region.

Figure 18.3 illustrates the result of the scan of a 5×5 input using a 3×3 filter with given values, and how the activation in the upper-right corner of the feature map results from the dot product of the flattened input region and the kernel:

Input Data	Filter Matrix (Kernel)	Feature Map																																			
<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr> </table>	1	1	1	0	0	0	1	1	1	0	0	0	1	1	1	0	0	1	1	0	0	1	1	0	0	*	<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> </table>	1	0	1	0	1	0	1	0	1	=
1	1	1	0	0																																	
0	1	1	1	0																																	
0	0	1	1	1																																	
0	0	1	1	0																																	
0	1	1	0	0																																	
1	0	1																																			
0	1	0																																			
1	0	1																																			
		<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>4</td><td>3</td><td>4</td></tr> <tr><td>2</td><td>4</td><td>3</td></tr> <tr><td>2</td><td>3</td><td>4</td></tr> </table>	4	3	4	2	4	3	2	3	4																										
4	3	4																																			
2	4	3																																			
2	3	4																																			

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}^T \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} = 4$$

Figure 18.3: From convolutions to a feature map

The most important aspect is that the **filter values are the parameters** of the convolutional layers, **learned from the data** during training to minimize the chosen loss function. In other words, CNNs learn useful feature representations by finding kernel values that activate input patterns that are most useful for the task at hand.

How to scan the input – strides and padding

The **stride** defines the step size used for scanning the input, that is, the number of pixels to shift horizontally and vertically. Smaller strides scan more (overlapping) areas but are computationally more expensive. Four options are commonly used when the filter does not fit the input perfectly and partially crosses the image boundary during the scan:

- **Valid convolution:** Discards scans where the image and filter do not perfectly match
- **Same convolution:** Zero-pads the input to produce a feature map of equal size
- **Full convolution:** Zero-pads the input so that each pixel is scanned an equal number of times, including pixels at the border (to avoid oversampling pixels closer to the center)
- **Causal:** Zero-pads the input only on the left so that the output does not depend on an input from a later period; maintains the temporal order for time-series data

The choices depend on the nature of the data and where useful features are most likely located. In combination with the number of depth slices, they determine the output size of the convolution stage. The Stanford lecture notes by Andrew Karpathy (see GitHub) contain helpful examples using NumPy.

Parameter sharing for robust features and fast computation

The location of salient features may vary due to distortion or shifts. Furthermore, elementary feature detectors are likely useful across the entire image. CNNs encode these assumptions by sharing or tying the weights for the filter in a given depth slice.

As a result, each depth slice specializes in a certain pattern and the number of parameters is further reduced. Weight sharing works less well, however, when images are spatially centered and key patterns are less likely to be uniformly distributed across the input area.

The detector stage – adding nonlinearity

The feature maps are usually passed through a nonlinear transformation. The **rectified linear unit (ReLU)** that we encountered in the last chapter is a common function for this purpose. ReLUs replace negative activations element-wise by zero and mitigate the risk of vanishing gradients found in other activation functions such as tanh (see *Chapter 17, Deep Learning for Trading*).

A popular alternative is the **softplus function**:

$$f(x) = \ln(1 + e^x)$$

In contrast to ReLU, it has a derivative everywhere, namely the sigmoid function that we used for logistic regression (see *Chapter 7, Linear Models – From Risk Factors to Return Forecasts*).

The pooling stage – downsampling the feature maps

The last stage of the convolutional layer may downsample the feature map's input representation to do the following:

- Reduce its dimensionality and prevent overfitting
- Lower the computational cost
- Enable basic translation invariance

This assumes that the precise location of the features is not only less important for identifying a pattern but can even be harmful because it will likely vary for different instances of the target. Pooling lowers the spatial resolution of the feature map as a simple way to render the location information less precise. However, this step is optional and many architectures use pooling only for some layers or not at all.

A common pooling operation is **max pooling**, which uses only the maximum activation value from (typically) non-overlapping subregions. For a small 4×4 feature map, for instance, 2×2 max pooling outputs the maximum for each of the four non-overlapping 2×2 areas. Less common pooling operators use the average or the median. Pooling does not add or learn new parameters but the size of the input window and possibly the stride are additional hyperparameters.

The evolution of CNN architectures – key innovations

Several CNN architectures have pushed performance boundaries over the past two decades by introducing important innovations. Predictive performance growth accelerated dramatically with the arrival of big data in the form of ImageNet (Fei-Fei 2015) with 14 million images assigned to 20,000 classes by humans via Amazon's Mechanical Turk. The **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** became the focal point of CNN progress around a slightly smaller set of 1.2 million images from 1,000 classes.

It is useful to be familiar with the **reference architectures** dominating these competitions for practical reasons. As we will see in the next section on working with CNNs for image data, they offer a good starting point for standard tasks. Moreover, **transfer learning** allows us to address many computer vision tasks by building on a successful architecture with pretrained weights. Transfer learning not only speeds up architecture selection and training but also enables successful applications on much smaller datasets.

In addition, many publications refer to these architectures, and they often serve as a basis for networks tailored to segmentation or localization tasks. We will further describe some landmark architectures in the section on image classification and transfer learning.

Performance breakthroughs and network size

The left side of *Figure 18.4* plots the top-1 accuracy against the computational cost of a variety of network architectures. It suggests a positive relationship between the number of parameters and performance, but also shows that the marginal benefit of more parameters declines and that architectural design and innovation also matter.

The right side plots the top-1 accuracy per parameter for all networks. Several new architectures target use cases on less powerful devices such as mobile phones. While they do not achieve state-of-the-art performance, they have found much more efficient implementations. See the resources on GitHub for more details on these architectures and the analysis behind these charts.

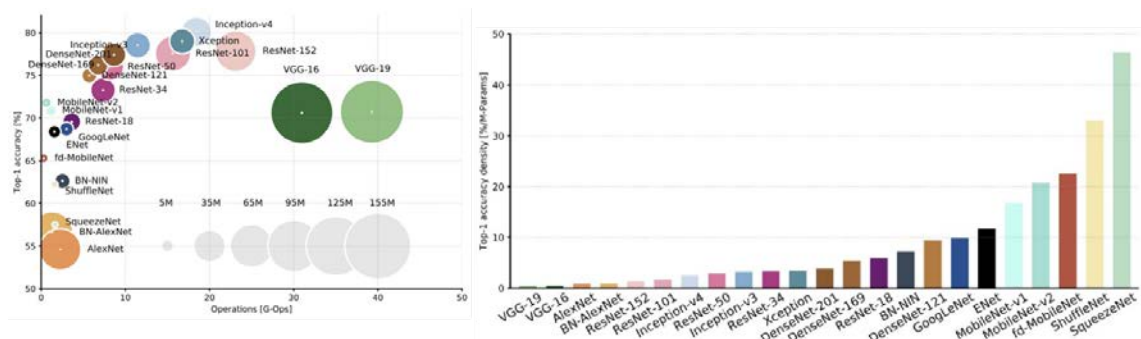


Figure 18.4: Predictive performance and computational complexity

Lessons learned

Some of the lessons learned from 20 years of CNN architecture developments, especially since 2012, include the following:

- **Smaller convolutional** filters perform better (possibly except at the first layer) because several small filters can substitute for a larger filter at a lower computational cost.
- **1×1 convolutions** reduce the dimensionality of feature maps so that the network can learn a larger number overall.
- **Skip connections** are able to create multiple paths through the network and enable the training of much higher-capacity CNNs.

CNNs for satellite images and object detection

In this section, we demonstrate how to solve key computer vision tasks such as image classification and object detection. As mentioned in the introduction and in *Chapter 3, Alternative Data for Finance – Categories and Use Cases*, image data can inform a trading strategy by providing clues about future trends, changing fundamentals, or specific events relevant to a target asset class or investment universe. Popular examples include exploiting satellite images for clues about the supply of agricultural commodities, consumer and economic activity, or the status of manufacturing or raw material supply chains. Specific tasks might include the following, for example:

- **Image classification:** Identifying whether cultivated land for certain crops is expanding, or predicting harvest quality and quantities
- **Object detection:** Counting the number of oil tankers on a certain transport route or the number of cars in a parking lot, or identifying the locations of shoppers in a mall

In this section, we'll demonstrate how to design CNNs to automate the extraction of such information, both from scratch using popular architectures and via transfer learning that fine-tunes pretrained weights to a given task. We'll also demonstrate how to detect objects in a given scene.

We will introduce key CNN architectures for these tasks, explain why they work well, and show how to train them using TensorFlow 2. We will also demonstrate how to source pretrained weights and fine-tune time. Unfortunately, satellite images with information directly relevant for a trading strategy are very costly to obtain and are not readily available. We will, however, demonstrate how to work with the EuroSat dataset to build a classifier that identifies different land uses. This brief introduction to CNNs for computer vision aims to demonstrate how to approach common tasks that you will likely need to tackle when aiming to design a trading strategy based on images relevant to the investment universe of your choice.

All the libraries we introduced in the last chapter provide support for convolutional layers; we'll focus on the Keras interface of TensorFlow 2. We are first going to illustrate the LeNet5 architecture using the MNIST handwritten digit dataset. Next, we'll demonstrate the use of data augmentation with AlexNet on CIFAR-10, a simplified version of the original ImageNet. Then we'll continue with transfer learning based on state-of-the-art architectures before we apply what we've learned to actual satellite images. We conclude with an example of object detection in real-life scenes.

LeNet5 – The first CNN with industrial applications

Yann LeCun, now the Director of AI Research at Facebook, was a leading pioneer in CNN development. In 1998, after several iterations starting in the 1980s, LeNet5 became the first modern CNN used in real-world applications that introduced several architectural elements still relevant today.

LeNet5 was published in a very instructive paper, *Gradient-Based Learning Applied to Document Recognition* (LeCun et al. 1989), that laid out many of the central concepts. Most importantly, it promoted the insight that convolutions with learnable filters are effective at extracting related features at multiple locations with few parameters. Given the limited computational resources at the time, efficiency was of paramount importance.

LeNet5 was designed to recognize the handwriting on checks and was used by several banks. It established a new benchmark for classification accuracy, with a result of 99.2 percent on the MNIST handwritten digit dataset. It consists of three convolutional layers, each containing a nonlinear tanh transformation, a pooling operation, and a fully connected output layer. Throughout the convolutional layers, the number of feature maps increases while their dimensions decrease. It has a total of 60,850 trainable parameters (Lecun et al. 1998).

"Hello World" for CNNs – handwritten digit classification

In this section, we'll implement a slightly simplified version of LeNet5 to demonstrate how to build a CNN using a TensorFlow implementation. The original MNIST dataset contains 60,000 grayscale images in 28×28 pixel resolution, each containing a single handwritten digit from 0 to 9. A good alternative is the more challenging but structurally similar Fashion MNIST dataset that we encountered in *Chapter 13, Data-Driven Risk Factors and Asset Allocation with Unsupervised Learning*. See the `digit_classification_with_lenet5` notebook for implementation details.

We can load it in Keras out of the box:

```
from tensorflow.keras.datasets import mnist
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train.shape, X_test.shape
((60000, 28, 28), (10000, 28, 28))
```

Figure 18.5 shows the first ten images in the dataset and highlights significant variation among instances of the same digit. On the right, it shows how the pixel values for an individual image range from 0 to 255:

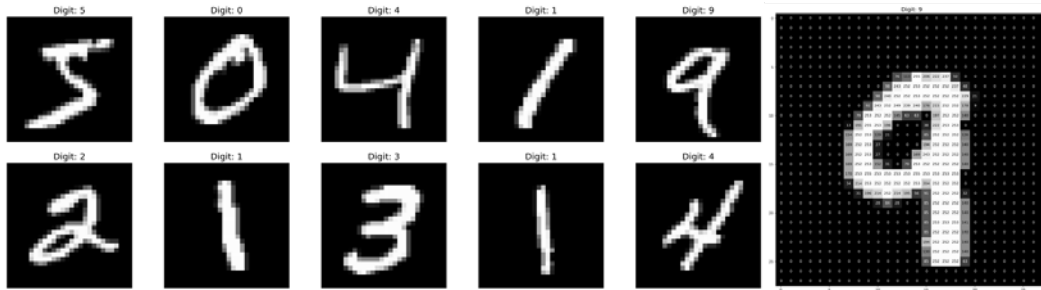


Figure 18.5: MNIST sample images

We rescale the pixel values to the range [0, 1] to normalize the training data and facilitate the backpropagation process and convert the data to 32-bit floats, which reduce memory requirements and computational cost while providing sufficient precision for our use case:

```
X_train = X_train.astype('float32')/255
X_test = X_test.astype('float32')/255
```

Defining the LeNet5 architecture

We can define a simplified version of LeNet5 that omits the original final layer containing radial basis functions as follows, using the default "valid" padding and single-step strides unless defined otherwise:

```
lenet5 = Sequential([
    Conv2D(filters=6, kernel_size=5, activation='relu',
           input_shape=(28, 28, 1), name='CONV1'),
    AveragePooling2D(pool_size=(2, 2), strides=(1, 1),
                     padding='valid', name='POOL1'),
    Conv2D(filters=16, kernel_size=(5, 5), activation='tanh', name='CONV2'),
    AveragePooling2D(pool_size=(2, 2), strides=(2, 2), name='POOL2'),
    Conv2D(filters=120, kernel_size=(5, 5), activation='tanh', name='CONV3'),
    Flatten(name='FLAT'),
    Dense(units=84, activation='tanh', name='FC6'),
    Dense(units=10, activation='softmax', name='FC7')
])
```

The summary indicates that the model thus defined has over 300,000 parameters:

Layer (type)	Output Shape	Param #
CONV1 (Conv2D)	(None, 24, 24, 6)	156
POOL1 (AveragePooling2D)	(None, 23, 23, 6)	0
CONV2 (Conv2D)	(None, 19, 19, 16)	2416
POOL2 (AveragePooling2D)	(None, 9, 9, 16)	0
CONV3 (Conv2D)	(None, 5, 5, 120)	48120

FLAT (Flatten)	(None, 3000)	0
FC6 (Dense)	(None, 84)	252084
FC7 (Dense)	(None, 10)	850
=====		
Total params:	303,626	
Trainable params:	303,626	

We compile with `sparse_crossentropy_loss`, which accepts integers rather than one-hot-encoded labels and the original stochastic gradient optimizer:

```
lenet5.compile(loss='sparse_categorical_crossentropy',
               optimizer='SGD',
               metrics=['accuracy'])
```

Training and evaluating the model

Now we are ready to train the model. The model expects four-dimensional input, so we reshape accordingly. We use the standard batch size of 32 and an 80:20 train-validation split. Furthermore, we leverage checkpointing to store the model weights if the validation error improves, and make sure the dataset is randomly shuffled. We also define an `early_stopping` callback to interrupt training once the validation accuracy no longer improves for 20 iterations:

```
lenet_history = lenet5.fit(X_train.reshape(-1, 28, 28, 1),
                           y_train,
                           batch_size=32,
                           epochs=100,
                           validation_split=0.2, # use 0 to train on all data
                           callbacks=[checkpointer, early_stopping],
                           verbose=1,
                           shuffle=True)
```

The training history records the last improvement after 81 epochs that take around 4 minutes on a single GPU. The test accuracy of this sample run is 99.09 percent, almost exactly the same result as for the original LeNet5:

```
accuracy = lenet5.evaluate(X_test.reshape(-1, 28, 28, 1), y_test, verbose=0)
[1]
print('Test accuracy: {:.2%}'.format(accuracy))
Test accuracy: 99.09%
```

For comparison, a simple two-layer feedforward network achieves "only" 97.04 percent test accuracy (see the notebook). The LeNet5 improvement on MNIST is, in fact, modest. Non-neural methods have also achieved classification accuracies greater than or equal to 99 percent, including K-nearest neighbors and support vector machines. CNNs really shine with more challenging datasets as we will see next.

AlexNet – reigniting deep learning research

AlexNet, developed by Alex Krizhevsky, Ilya Sutskever, and Geoff Hinton at the University of Toronto, dramatically reduced the error rate and significantly outperformed the runner-up at the 2012 ILSVRC, achieving a top-5 error of 16 percent versus 26 percent (Krizhevsky, Sutskever, and Hinton 2012). This breakthrough triggered a renaissance in ML research and put deep learning for computer vision firmly on the global technology map.

The AlexNet architecture is similar to LeNet, but much deeper and wider. It is often credited with discovering **the importance of depth** with around 60 million parameters, exceeding LeNet5 by a factor of 1,000, a testament to increased computing power, especially the use of GPUs, and much larger datasets.

It included convolutions stacked on top of each other rather than combining each convolution with a pooling stage, and successfully used dropout for regularization and ReLU for efficient nonlinear transformations. It also employed data augmentation to increase the number of training samples, added weight decay, and used a more efficient implementation of convolutions. It also accelerated training by distributing the network over two GPUs.

The notebook `image_classification_with_alexnet.ipynb` has a slightly simplified version of AlexNet tailored to the CIFAR-10 dataset that contains 60,000 images from 10 of the original 1,000 classes. It has been compressed to a 32×32 pixel resolution from the original 224×224 , but still has three color channels.

See the notebook `image_classification_with_alexnet` for implementation details; we will skip over some repetitive steps here.

Preprocessing CIFAR-10 data using image augmentation

CIFAR-10 can also be downloaded using TensorFlow's Keras interface, and we rescale the pixel values and one-hot encode the ten class labels as we did with MNIST in the previous section.

We first train a two-layer feedforward network on 50,000 training samples for 45 epochs to achieve a test accuracy of 45.78 percent. We also experiment with a three-layer convolutional net with over 528,000 parameters that achieves 74.51 percent test accuracy (see the notebook).

A common trick to enhance performance is to artificially increase the size of the training set by creating synthetic data. This involves randomly shifting or horizontally flipping the image or introducing noise into the image. TensorFlow includes an `ImageDataGenerator` class for this purpose. We can configure it and fit the training data as follows:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(
    width_shift_range=0.1,    # randomly horizontal shift
    height_shift_range=0.1,  # randomly vertical shift
    horizontal_flip=True)    # randomly horizontal flip
datagen.fit(X_train)
```

The result shows how the augmented images (in low 32×32 resolution) have been altered in various ways as expected:



Figure 18.6: Original and augmented samples

The test accuracy for the three-layer CNN improves modestly to 76.71 percent after training on the larger, augmented data.

Defining the model architecture

We need to adapt the AlexNet architecture to the lower dimensionality of CIFAR-10 images relative to the ImageNet samples used in the competition. To this end, we use the original number of filters but make them smaller (see the notebook for implementation details).

The summary (see the notebook) shows the five convolutional layers followed by two fully connected layers with frequent use of batch normalization, for a total of 21.5 million parameters.

Comparing AlexNet performance

In addition to AlexNet, we trained a 2-layer feedforward NN and a 3-layer CNN, the latter with and without image augmentation. After 100 epochs (with early stopping if the validation accuracy does not improve for 20 rounds), we obtain the cross-validation trajectories and test accuracy for the four models, as displayed in *Figure 18.7*:

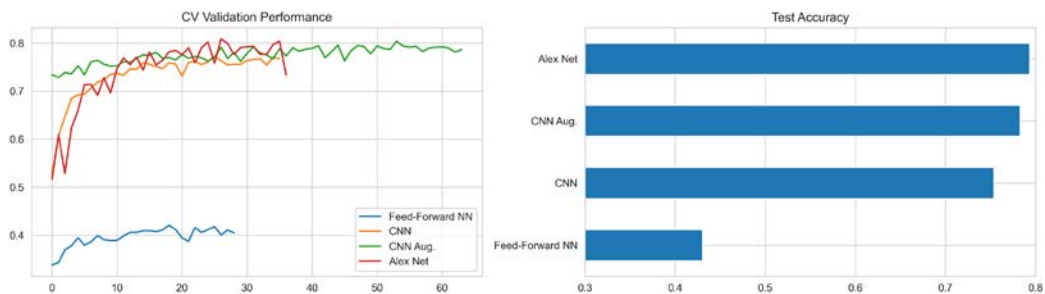


Figure 18.7: Validation performance and test accuracy on CIFAR-10

AlexNet achieves the highest test accuracy with 79.33 percent after some 35 epochs, closely followed by the shallower CNN with augmented images at 78.29 percent that trains for longer due to the larger dataset. The feedforward NN performs much worse than on MNIST on this more complex dataset, with a test accuracy of 43.05 percent.

Transfer learning – faster training with less data

In practice, sometimes we do not have enough data to train a CNN from scratch with random initialization. **Transfer learning** is an ML technique that repurposes a model trained on one set of data for another task. Naturally, it works if the learning from the first task carries over to the task of interest. If successful, it can lead to better performance and faster training that requires less labeled data than training a neural network from scratch on the target task.

Alternative approaches to transfer learning

The transfer learning approach to CNN relies on pretraining on a very large dataset like ImageNet. The goal is for the convolutional filters to extract a feature representation that generalizes to new images. In a second step, it leverages the result to either initialize and retrain a new CNN or use it as input to a new network that tackles the task of interest.

As discussed, CNN architectures typically use a sequence of convolutional layers to detect hierarchical patterns, adding one or more fully connected layers to map the convolutional activations to the outcome classes or values. The output of the last convolutional layer that feeds into the fully connected part is called the bottleneck features. We can use the **bottleneck features** of a pretrained network as inputs into a new fully connected network, usually after applying a ReLU activation function.

In other words, we freeze the convolutional layers and **replace the dense part of the network**. An additional benefit is that we can then use inputs of different sizes because it is the dense layers that constrain the input size.

Alternatively, we can use the bottleneck features as **inputs into a different machine learning algorithm**. In the AlexNet architecture, for instance, the bottleneck layer computes a vector with 4,096 entries for each 224×224 input image. We then use this vector as features for a new model.

We also can go a step further and not only replace and retrain the final layers using new data but also **fine-tune the weights of the pretrained CNN**. To achieve this, we continue training, either only for later layers while freezing the weights of some earlier layers, or for all layers. The motivation is presumably to preserve more generic patterns learned by lower layers, such as edge or color blob detectors, while allowing later layers of the CNN to adapt to the details of a new task. ImageNet, for example, contains a wide variety of dog breeds, which may lead to feature representations specifically useful for differentiating between these classes.

Building on state-of-the-art architectures

Transfer learning permits us to leverage top-performing architectures without incurring the potentially fairly GPU- and data-intensive training. We briefly outline the key characteristics of a few additional popular architectures that are popular starting points.

VGGNet – more depth and smaller filters

The runner-up in ILSVRC 2014 was developed by Oxford University's Visual Geometry Group (VGG, Simonyan 2015). It demonstrated the effectiveness of **much smaller** 3×3 **convolutional filters** combined in sequence and reinforced the importance of depth for strong performance. VGG16 contains 16 convolutional and fully connected layers that only perform 3×3 convolutions and 2×2 pooling (see *Figure 18.5*).

VGG16 has **140 million parameters** that increase the computational costs of training and inference as well as the memory requirements. However, most parameters are in the fully connected layers that were since discovered not to be essential so that removing them greatly reduces the number of parameters without negatively impacting performance.

GoogLeNet – fewer parameters through Inception

Christian Szegedy at Google reduced the computational costs using more efficient CNN implementations to facilitate practical applications at scale. The resulting GoogLeNet (Szegedy et al. 2015) won the ILSVRC 2014 with only 4 million parameters due to the **Inception module**, compared to AlexNet's 60 million and VGG16's 140 million.

The Inception module builds on the **network-in-network concept** that uses 1×1 convolutions to compress a deep stack of convolutional filters and thus reduce the cost of computation. The module uses parallel 1×1 , 3×3 , and 5×5 filters, combining the latter two with 1×1 convolutions to reduce the dimensionality of the filters passed in by the previous layer.

In addition, it uses average pooling instead of fully connected layers on top of the convolutional layers to eliminate many of the less impactful parameters. There have been several enhanced versions, most recently Inception-v4.

ResNet – shortcut connections beyond human performance

The **residual network (ResNet)** architecture was developed at Microsoft and won the ILSVRC 2015. It pushed the top-5 error to 3.7 percent, below the level of human performance on this task of around 5 percent (He et al. 2015).

It introduces identity shortcut connections that skip several layers and overcome some of the challenges of training deep networks, enabling the use of hundreds or even over a thousand layers. It also heavily uses batch normalization, which was shown to allow higher learning rates and be more forgiving about weight initialization. The architecture also omits the fully connected final layers.

As mentioned in the last chapter, the training of deep networks faces the notorious vanishing gradient challenge: as the gradient propagates to earlier layers, repeated multiplication of small weights risks shrinking the gradient toward zero. Hence, increasing depth may limit learning.

The shortcut connection that skips two or more layers has become one of the most popular developments in CNN architectures and triggered numerous research efforts to refine and explain its performance. See the references on GitHub for additional information.

Transfer learning with VGG16 in practice

Modern CNNs can take weeks to train on multiple GPUs on ImageNet, but fortunately, many researchers share their final weights. TensorFlow 2, for example, contains pretrained models for several of the reference architectures discussed previously, namely VGG16 and its larger version, VGG19, ResNet50, InceptionV3, and InceptionResNetV2, as well as MobileNet, DenseNet, NASNet, and MobileNetV2.

How to extract bottleneck features

The notebook `bottleneck_features.ipynb` illustrates how to download the pretrained VGG16 model, either with the final layers to generate predictions or without the final layers, as illustrated in *Figure 18.8*, to extract the outputs produced by the bottleneck features:

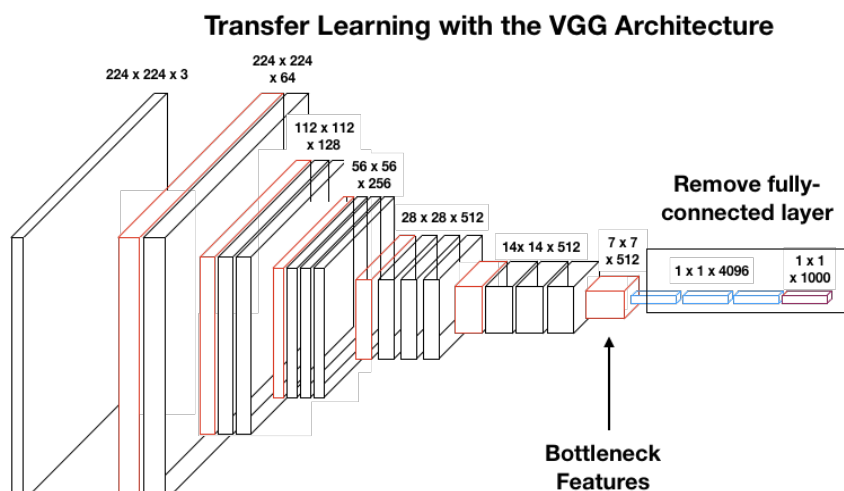


Figure 18.8: The VGG16 architecture

TensorFlow 2 makes it very straightforward to download and use pretrained models:

```
from tensorflow.keras.applications.vgg16 import VGG16
vgg16 = VGG16()
vgg16.summary()
Layer (type)                Output Shape                Param #
input_1 (InputLayer)        (None, 224, 224, 3)        0
... several layers omitted...
```

block5_conv4 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102764544
fc2 (Dense)	(None, 4096)	16781312
predictions (Dense)	(None, 1000)	4097000
Total params: 138,357,544		
Trainable params: 138,357,544		

You can use this model for predictions like any other Keras model: we pass in seven sample images and obtain class probabilities for each of the 1,000 ImageNet categories:

```
y_pred = vgg16.predict(img_input)
Y_pred.shape
(7, 1000)
```

To exclude the fully connected layers, just add the keyword `include_top=False`. Predictions are now output by the final convolutional layer `block5_pool` and match this layer's shape:

```
vgg16 = VGG16(include_top=False)
vgg16.predict(img_input).shape
(7, 7, 7, 512)
```

By omitting the fully connected layers and keeping only the convolutional modules, we are no longer forced to use a fixed input size for the model such as the original 224×224 ImageNet format. Instead, we can adapt the model to arbitrary input sizes.

How to fine-tune a pretrained model

We will demonstrate how to freeze some or all of the layers of a pretrained model and continue training using a new fully-connected set of layers and data with a different format (see the notebook `transfer_learning.ipynb` for code examples, adapted from a TensorFlow 2 tutorial).

We use the VGG16 weights, pretrained on ImageNet with TensorFlow's built-in cats versus dogs images (see the notebook on how to source the dataset).

Preprocessing resizes all images to 160×160 pixels. We indicate the new input size as we instantiate the pretrained VGG16 instance and then freeze all weights:

```
vgg16 = VGG16(input_shape=IMG_SHAPE, include_top=False, weights='imagenet')
vgg16.trainable = False
vgg16.summary()
```

Layer (type)	Output Shape	Param #
... omitted layers...		
block5_conv3 (Conv2D)	(None, 10, 10, 512)	2359808
block5_pool (MaxPooling2D)	(None, 5, 5, 512)	0

```
Total params: 14,714,688
Trainable params: 0
Non-trainable params: 14,714,688
```

The shape of the model output for 32 sample images now matches that of the last convolutional layer in the headless model:

```
feature_batch = vgg16(image_batch)
Feature_batch.shape
TensorShape([32, 5, 5, 512])
```

We can append new layers to the headless model using either the Sequential or the Functional API. For the Sequential API, adding GlobalAveragePooling2D, Dense, and Dropout layers works as follows:

```
global_average_layer = GlobalAveragePooling2D()
dense_layer = Dense(64, activation='relu')
dropout = Dropout(0.5)
prediction_layer = Dense(1, activation='sigmoid')
seq_model = tf.keras.Sequential([vgg16,
                                  global_average_layer,
                                  dense_layer,
                                  dropout,
                                  prediction_layer])
seq_model.compile(loss = tf.keras.losses.BinaryCrossentropy(from_logits=True),
                  optimizer = 'Adam',
                  metrics=["accuracy"])
```

We set `from_logits=True` for the BinaryCrossentropy loss because the model provides a linear output. The summary shows how the new model combines the pretrained VGG16 convolutional layers and the new final layers:

```
seq_model.summary()
Layer (type)                                     Output Shape                                     Param #
vgg16 (Model)                                    (None, 5, 5, 512)                               14714688
global_average_pooling2d (GlobalAveragePooling2D) (None, 512)                                       0
dense_7 (Dense)                                  (None, 64)                                       32832
dropout_3 (Dropout)                              (None, 64)                                       0
dense_8 (Dense)                                  (None, 1)                                        65
Total params: 14,747,585
Trainable params: 11,831,937
Non-trainable params: 2,915,648
```

See the notebook for the Functional API version.

Prior to training the new final layer, the pretrained VGG16 delivers a validation accuracy of 48.75 percent. Now we proceed to train the model for 10 epochs as follows, adjusting only the final layer weights:

```
history = transfer_model.fit(train_batches,
                             epochs=initial_epochs,
                             validation_data=validation_batches)
```

10 epochs boost validation accuracy above 94 percent. To fine-tune the model, we can unfreeze the VGG16 models and continue training. Note that you should only do so after training the new final layers: randomly initialized classification layers will likely produce large gradient updates that can eliminate the pretraining results.

To unfreeze parts of the model, we select a layer, after which we set the weights to trainable; in this case, layer 12 of the total 19 layers in the VGG16 architecture:

```
vgg16.trainable = True
len(vgg16.layers)
19
# Fine-tune from this layer onward
start_fine_tuning_at = 12
# Freeze all the layers before the 'fine_tune_at' layer
for layer in vgg16.layers[:start_fine_tuning_at]:
    layer.trainable = False
```

Now just recompile the model and continue training for up to 50 epochs using early stopping, starting in epoch 10 as follows:

```
fine_tune_epochs = 50
total_epochs = initial_epochs + fine_tune_epochs
history_fine_tune = transfer_model.fit(train_batches,
                                       epochs=total_epochs,
                                       initial_epoch=history.epoch[-1],
                                       validation_data=validation_batches,
                                       callbacks=[early_stopping])
```

Figure 18.9 shows how the validation accuracy increases substantially, reaching 97.89 percent after another 22 epochs:

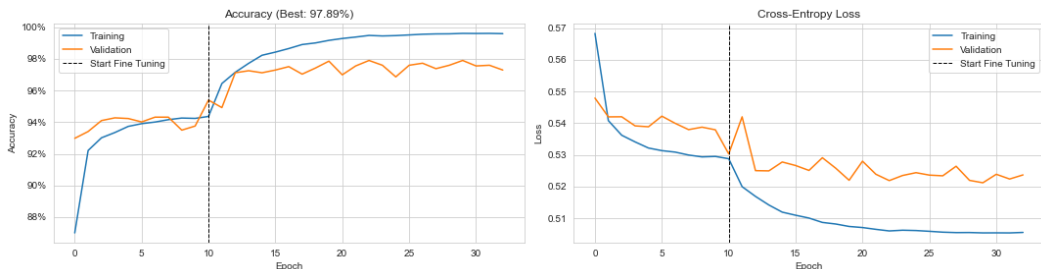


Figure 18.9: Cross-validation performance: accuracy and cross-entropy loss

Transfer learning is an important technique when training data is limited as is very often the case in practice. While cats and dogs are unlikely to produce tradeable signals, transfer learning could certainly help improve the accuracy of predictions on a relevant alternative dataset, such as the satellite images that we'll tackle next.

Classifying satellite images with transfer learning

Satellite images figure prominently among alternative data (see *Chapter 3, Alternative Data for Finance – Categories and Use Cases*). For instance, commodity traders may rely on satellite images to predict the supply of certain crops or resources by monitoring, activity on farms, at mining sites, or oil tanker traffic.

The EuroSat dataset

To illustrate working with this type of data, we load the EuroSat dataset included in the TensorFlow 2 datasets (Helber et al. 2019). The EuroSat dataset includes around 27,000 images in 64×64 format that represent 10 different types of land uses. *Figure 18.10* displays an example for each label:



Figure 18.10: Ten types of land use contained in the dataset

A time series of similar data could be used to track the relative sizes of cultivated, industrial, and residential areas or the status of specific crops to predict harvest quantities or quality, for example, for wine.

Fine-tuning a very deep CNN – DenseNet201

Huang et al. (2018) developed a new architecture dubbed **densely connected** based on the insight that CNNs can be deeper, more accurate, and more efficient to train if they contain shorter connections between layers close to the input and those close to the output.

One architecture, labeled **DenseNet201**, connects each layer to every other layer in a feedforward fashion. It uses the feature maps of all preceding layers as inputs, while each layer's own feature maps become inputs into all subsequent layers.

We download the DenseNet201 architecture from `tensorflow.keras.applications` and replace its final layers with the following dense layers interspersed with batch normalization to mitigate exploding or vanishing gradients in this very deep network with over 700 layers:

Layer (type)	Output Shape	Param #
densenet201 (Model)	(None, 1920)	18321984
batch_normalization (Batch Normalization)	(None, 1920)	7680
dense (Dense)	(None, 2048)	3934208
batch_normalization_1 (Batch Normalization)	(None, 2048)	8192
dense_1 (Dense)	(None, 2048)	4196352
batch_normalization_2 (Batch Normalization)	(None, 2048)	8192
dense_2 (Dense)	(None, 2048)	4196352
batch_normalization_3 (Batch Normalization)	(None, 2048)	8192
dense_3 (Dense)	(None, 2048)	4196352
batch_normalization_4 (Batch Normalization)	(None, 2048)	8192
dense_4 (Dense)	(None, 10)	20490
Total params: 34,906,186		
Trainable params: 34,656,906		
Non-trainable params: 249,280		

Model training and results evaluation

We use 10 percent of the training images for validation purposes and achieve the best out-of-sample classification accuracy of 97.96 percent after 10 epochs. This exceeds the performance cited in the original paper for the best-performing ResNet-50 architecture with a 90-10 split.

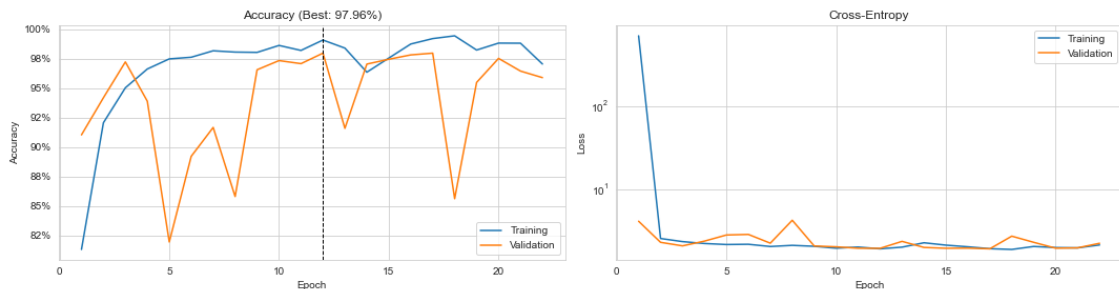


Figure 18.11: Cross-validation performance

There would likely be additional performance gains from augmenting the relatively small training set.

Object detection and segmentation

Image classification is a fundamental computer vision task that requires labeling an image based on certain objects it contains. Many practical applications, including investment and trading strategies, require additional information:

- The **object detection** task requires not only the identification but also the spatial location of all objects of interest, typically using bounding boxes. Several algorithms have been developed to overcome the inefficiency of brute-force sliding-window approaches, including region proposal methods (R-CNN; see for example Ren et al. 2015) and the **You Only Look Once (YOLO)** real-time object detection algorithm (Redmon 2016).
- The **object segmentation** task goes a step further and requires a class label and an outline of every object in the input image. This may be useful to count objects such as oil tankers, individuals, or cars in an image and evaluate a level of activity.
- **Semantic segmentation**, also called scene parsing, makes dense predictions to assign a class label to each pixel in the image. As a result, the image is divided into semantic regions and each pixel is assigned to its enclosing object or region.

Object detection requires the ability to distinguish between several classes of objects and to decide how many and which of these objects are present in an image.

Object detection in practice

A prominent example is Ian Goodfellow's identification of house numbers from Google's **Street View House Numbers (SVHN)** dataset (Goodfellow 2014). It requires the model to identify the following:

- How many of up to five digits make up the house number
- The correct digit for each component
- The proper order of the constituent digits

We will show how to preprocess the irregularly shaped source images, adapt the VGG16 architecture to produce multiple outputs, and train the final layer, before fine-tuning the pretrained weights to address the task.

Preprocessing the source images

The notebook `svhn_preprocessing.ipynb` contains code to produce a simplified, cropped dataset that uses bounding box information to create regularly shaped 32×32 images containing the digits; the original images are of arbitrary shape (Netzer 2011).



Figure 18.12: Cropped sample images of the SVHN dataset

The SVHN dataset contains house numbers with up to five digits and uses the class 10 if a digit is not present. However, since there are very few examples with five digits, we limit the images to those including up to four digits only.

Transfer learning with a custom final layer

The notebook `svhn_object_detection.ipynb` illustrates how to apply transfer learning to a deep CNN based on the VGG16 architecture, as outlined in the previous section. We will describe how to create new final layers that produce several outputs to meet the three SVHN task objectives, including one prediction of how many digits are present, and one for the value of each digit in the order they appear.

The best-performing architecture on the original dataset has eight convolutional layers and two final fully connected layers. We will use **transfer learning**, departing from the VGG16 architecture. As before, we import the VGG16 network pretrained on ImageNet weights, remove the layers after the convolutional blocks, freeze the weights, and create new dense and predictive layers as follows using the Functional API:

```
vgg16 = VGG16(input_shape=IMG_SHAPE, include_top=False, weights='imagenet')
vgg16.trainable = False
x = vgg16.output
x = Flatten()(x)
x = BatchNormalization()(x)
x = Dense(256)(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = Dense(128)(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
n_digits = Dense(SEQ_LENGTH, activation='softmax', name='n_digits')(x)
digit1 = Dense(N_CLASSES-1, activation='softmax', name='d1')(x)
digit2 = Dense(N_CLASSES, activation='softmax', name='d2')(x)
digit3 = Dense(N_CLASSES, activation='softmax', name='d3')(x)
digit4 = Dense(N_CLASSES, activation='softmax', name='d4')(x)
predictions = Concatenate()([n_digits, digit1, digit2, digit3, digit4])
```


The prediction layer combines the four-class output for the number of digits `n_digits` with four outputs that predict which digit is present at that position.

Creating a custom loss function and evaluation metrics

The custom output requires us to define a loss function that captures how well the model is meeting its objective. We would also like to measure accuracy in a way that reflects predictive accuracy tailored to the specific labels.

For the custom loss, we average the cross-entropy over the five categorical outputs, namely the number of digits and their respective values:

```
def weighted_entropy(y_true, y_pred):
    cce = tf.keras.losses.SparseCategoricalCrossentropy()
    n_digits = y_pred[:, :SEQ_LENGTH]
    digits = {}
    for digit, (start, end) in digit_pos.items():
        digits[digit] = y_pred[:, start:end]
    return (cce(y_true[:, 0], n_digits) +
            cce(y_true[:, 1], digits[1]) +
            cce(y_true[:, 2], digits[2]) +
            cce(y_true[:, 3], digits[3]) +
            cce(y_true[:, 4], digits[4])) / 5
```

To measure predictive accuracy, we compare the five predictions with the corresponding label values and average the share of correct matches over the batch of samples:

```
def weighted_accuracy(y_true, y_pred):
    n_digits_pred = K.argmax(y_pred[:, :SEQ_LENGTH], axis=1)

    digit_preds = {}
    for digit, (start, end) in digit_pos.items():
        digit_preds[digit] = K.argmax(y_pred[:, start:end], axis=1)
    preds = tf.dtypes.cast(tf.stack((n_digits_pred,
                                     digit_preds[1],
                                     digit_preds[2],
                                     digit_preds[3],
                                     digit_preds[4]), axis=1), tf.float32)

    return K.mean(K.sum(tf.dtypes.cast(K.equal(y_true, preds), tf.int64),
axis=1) / 5)
```

Finally, we integrate the base and final layers and compile the model with the custom loss and accuracy metric as follows:

```
model = Model(inputs=vgg16.input, outputs=predictions)
model.compile(optimizer='adam',
              loss=weighted_entropy,
              metrics=[weighted_accuracy])
```

Fine-tuning the VGG16 weights and final layer

We train the new final layers for 14 periods and continue fine-tuning all VGG16 weights, as in the previous section, for another 23 epochs (using early stopping in both cases).

The following charts show the training and validation accuracy and the loss over the entire training period. As we unfreeze the VGG16 weights after the initial training period, the accuracy drops and then improves, achieving a validation performance of 94.52 percent:

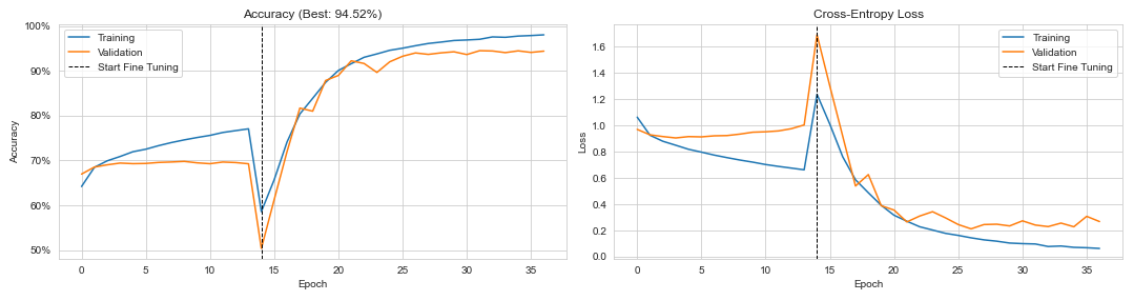


Figure 18.13: Cross-validation performance

See the notebook for additional implementation details and an evaluation of the results.

Lessons learned

We can achieve decent levels of accuracy using only the small training set. However, state-of-the-art performance achieves an error rate of only 1.02 percent (<https://benchmarks.ai/svhn>). To get closer, the most important step is to increase the amount of training data.

There are two easy ways to accomplish this: we can include the larger number of samples included in the **extra** dataset, and we can use image augmentation (see the *AlexNet: reigniting deep learning research* section). The currently best-performing approach relies heavily on augmentation learned from data (Cubuk 2019).

CNNs for time-series data – predicting returns

CNNs were originally developed to process image data and have achieved superhuman performance on various computer vision tasks. As discussed in the first section, time-series data has a grid-like structure similar to that of images, and CNNs have been successfully applied to one-, two- and three-dimensional representations of temporal data.

The application of CNNs to time series will most likely bear fruit if the data meets the model's key assumption that local patterns or relationships help predict the outcome. In the time-series context, local patterns could be autocorrelation or similar non-linear relationships at relevant intervals. Along the second and third dimensions, local patterns imply systematic relationships among different components of a multivariate series or among these series for different tickers. Since locality matters, it is important that the data is organized accordingly, in contrast to feed-forward networks where shuffling the elements of any dimension does not negatively affect the learning process.

In this section, we provide a relatively simple example using a one-dimensional convolution to model an autoregressive process (see *Chapter 9, Time-Series Models for Volatility Forecasts and Statistical Arbitrage*) that predicts future returns based on lagged returns. Then we replicate a recent research paper that achieved good results by formatting multivariate time-series data like images to predict returns. We will also develop and test a trading strategy based on the signals contained in the predictions.

An autoregressive CNN with 1D convolutions

We will introduce the time series use case for CNN using a univariate autoregressive asset return model. More specifically, the model receives the most recent 12 months of returns and uses a single layer of one-dimensional convolutions to predict the subsequent month.

The requisite steps are as follows:

1. Creating the rolling 12 months of lagged returns and corresponding outcomes
2. Defining the model architecture
3. Training the model and evaluating the results

In the following sections, we'll describe each step in turn; the notebook `time_series_prediction` contains the code samples for this section.

Preprocessing the data

First, we'll select the adjusted close price for all Quandl Wiki stocks since 2000 as follows:

```
prices = (pd.read_hdf('../data/assets.h5', 'quandl/wiki/prices')
          .adj_close
          .unstack().loc['2000':])
prices.info()
DatetimeIndex: 2896 entries, 2007-01-01 to 2018-03-27
Columns: 3199 entries, A to ZUMZ
```

Next, we resample the price data to month-end frequency, compute returns, and set monthly returns over 100 percent to missing as they likely represent data errors. Then we drop tickers with missing observations, retaining 1,511 stocks with 215 observations each:

```
returns = (prices
           .resample('M')
           .last()
           .pct_change()
           .dropna(how='all')
           .loc['2000': '2017']
           .dropna(axis=1)
           .sort_index(ascending=False))

# remove outliers likely representing data errors
returns = returns.where(returns<1).dropna(axis=1)
returns.info()
DatetimeIndex: 215 entries, 2017-12-31 to 2000-02-29
Columns: 1511 entries, A to ZQK
```

To create the rolling series of 12 lagged monthly returns with their corresponding outcome, we iterate over rolling 13-month slices and append the transpose of each slice to a list after assigning the outcome date to the index. After completing the loop, we concatenate the DataFrames in the list as follows:

```
n = len(returns)
nlags = 12
lags = list(range(1, nlags + 1))
cnn_data = []
for i in range(n-nlags-1):
    df = returns.iloc[i:i+nlags+1]          # select outcome and lags
    date = df.index.max()                   # use outcome date
    cnn_data.append(df.reset_index(drop=True) # append transposed series
                    .transpose()
                    .assign(date=date)
                    .set_index('date', append=True)
                    .sort_index(1, ascending=True))
```

```
cnn_data = (pd.concat(cnn_data)
            .rename(columns={0: 'label'})
            .sort_index())
```

We end up with over 305,000 pairs of outcomes and lagged returns for the 2001-2017 period:

```
cnn_data.info(null_counts=True)
MultiIndex: 305222 entries, ('A', Timestamp('2001-03-31 00:00:00')) to
              ('ZQK', Timestamp('2017-12-31 00:00:00'))
Data columns (total 13 columns):
...
```

When we compute the information coefficient for each lagged return and the outcome, we find that only lag 5 is not statistically significant:

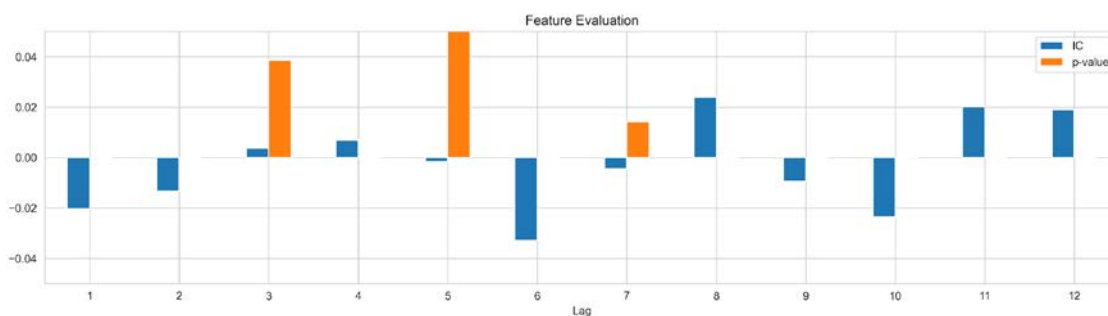


Figure 18.14: Information coefficient with respect to forward return by lag

Defining the model architecture

Now we'll define the model architecture using TensorFlow's Keras interface. We combine a one-dimensional convolutional layer with max pooling and batch normalization to produce a real-valued scalar output:

```
model = Sequential([Conv1D(filters=32,
                           kernel_size=4,
                           activation='relu',
                           padding='causal',
                           input_shape=(12, 1),
                           use_bias=True,
                           kernel_regularizer=regularizers.l1_l2(l1=1e-5,
                                                                    l2=1e-5)),
                   MaxPooling1D(pool_size=4),
                   Flatten(),
                   BatchNormalization(),
                   Dense(1, activation='linear')])
```

The one-dimensional convolution computes the sliding dot product of a (regularized) vector of length 4 with each input sequence of length 12, using causal padding to maintain the temporal order (see the *How to scan the input: strides and padding* section). The resulting 32 feature maps have the same length, 12, as the input that max pooling in groups of size 4 reduces to 32 vectors of length 3.

The model outputs the weighted average plus the bias of the flattened and normalized single vector of length 96, and has 449 trainable parameters:

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 12, 32)	160
max_pooling1d (MaxPooling1D)	(None, 3, 32)	0
flatten (Flatten)	(None, 96)	0
batch_normalization (BatchNo	(None, 96)	384
dense (Dense)	(None, 1)	97
Total params: 641		
Trainable params: 449		
Non-trainable params: 192		

The notebook wraps the model generation and subsequent compilation into a `get_model()` function that parametrizes the model configuration to facilitate experimentation.

Model training and performance evaluation

We train the model on five years of data for each ticker to predict the first month after this period and repeat this procedure 36 times using the `MultipleTimeSeriesCV` we developed in *Chapter 7, Linear Models – From Risk Factors to Return Forecasts*. See the notebook for the training loop that follows the pattern demonstrated in the previous chapter.

We use early stopping after five epochs to simplify the exposition, resulting in a positive bias so that the results have only illustrative character. Training length varies from 1 to 27 epochs, with a median of 5 epochs, which demonstrates that the model can often only learn very limited amounts of systematic information from the past returns. Thus cherry-picking the results yields a cumulative average information coefficient of around 4, as shown in *Figure 18.15*:



Figure 18.15: (Biased) out-of-sample information coefficients for best epochs

We'll now proceed to a more complex example of using CNNs for multiple time-series data.

CNN-TA – clustering time series in 2D format

To exploit the grid-like structure of time-series data, we can use CNN architectures for univariate and multivariate time series. In the latter case, we consider different time series as channels, similar to the different color signals.

An alternative approach converts a time series of alpha factors into a two-dimensional format to leverage the ability of CNNs to detect local patterns. Sezer and Ozbayoglu (2018) propose **CNN-TA**, which computes 15 technical indicators for different intervals and uses hierarchical clustering (see *Chapter 13, Data-Driven Risk Factors and Asset Allocation with Unsupervised Learning*) to locate indicators that behave similarly close to each other in a two-dimensional grid.

The authors train a CNN similar to the CIFAR-10 example we used earlier to predict whether to buy, hold, or sell an asset on a given day. They compare the CNN performance to "buy-and-hold" and other models and find that it outperforms all alternatives using daily price series for Dow 30 stocks and the nine most-traded ETFs over the 2007-2017 time period.

In this section, we experiment with this approach using daily US equity price data and demonstrate how to compute and convert a similar set of indicators into image format. Then we train a CNN to predict daily returns and evaluate a simple long-short strategy based on the resulting signals.

Creating technical indicators at different intervals

We first select a universe of the 500 most-traded US stocks from the Quandl Wiki dataset by dollar volume for rolling five-year periods for 2007-2017. See the notebook `engineer_cnn_features.ipynb` for the code examples in this section and some additional implementation details.

Our features consist of 15 technical indicators and risk factors that we compute for 15 different intervals and then arrange them in a 15×15 grid. The following table lists some of the technical indicators; in addition, we follow the authors in using the following metrics (see the *Appendix* for additional information):

- **Weighted and exponential moving averages (WMA and EMA)** of the close price
- **Rate of change (ROC)** of the close price
- **Chande Momentum Oscillator (CMO)**
- **Chaikin A/D Oscillators (ADOSC)**
- **Average Directional Movement Index (ADX)**

Indicator	Name	Formula
Relative Strength Index (RSI)	Oscillates in [0, 100] range; below 30: oversold, over 70: overbought	See Chapter 4
Williams %R	Momentum-based in [-100, 0] range, below -80: oversold, above -20: overbought	$R = \frac{\max(\text{high}) - \text{close}}{\max(\text{high}) - \min(\text{low})}$
Bollinger Bands	20-day moving average plus/minus daily standard deviation; prices above/below these bands indicate overbought/sold	See chapter 4.
Normalized Average True Range (NATR)	Avg. true range: max of current high-low, current high-prev.close or absolute of prev. close - current low, averaged over t days.	$\text{NATR} = \frac{\text{ATR}(t)}{\text{Close}}$
Percentage Price Oscillator (PPO)	Momentum: compares two exponential moving averages (EMA) in percentage terms	$\text{PPO} = \frac{\text{EMA}_{12} - \text{EMA}_{26}}{\text{EMA}_{26}}$
Commodity Channel Index (CCI)	Momentum-based: difference between current and simple moving average (SMA) of the historical average price, normalized by their mean difference	$\text{CCI} = \frac{\rho^{\text{hist}} - \text{SMA}(\rho^{\text{hist}})}{0.15 \times \sum_{i=1}^P (\rho^{\text{hist}} - \text{SMA}(\rho^{\text{hist}})) / P}$

Figure 8.16: Technical indicators

For each indicator, we vary the time period from 6 to 20 to obtain 15 distinct measurements. For example, the following code example computes the **relative strength index (RSI)**:

```
T = list(range(6, 21))
for t in T:
    universe[f'{t:02}_RSI'] = universe.groupby(level='symbol').close.
    apply(RSI, timeperiod=t)
```

For the **Normalized Average True Range (NATR)** that requires several inputs, the computation works as follows:

```
for t in T:
    universe[f'{t:02}_NATR'] = universe.groupby(
        level='symbol', group_keys=False).apply(
            lambda x: NATR(x.high, x.low, x.close, timeperiod=t))
```

See the TA-Lib documentation for further details.

Computing rolling factor betas for different horizons

We also use **five Fama-French risk factors** (Fama and French, 2015; see *Chapter 4, Financial Feature Engineering – How to Research Alpha Factors*). They reflect the sensitivity of a stock's returns to factors consistently demonstrated to impact equity returns. We capture these factors by computing the coefficients of a rolling OLS regression of a stock's daily returns on the returns of portfolios designed to reflect the underlying drivers:

- **Equity risk premium:** Value-weighted returns of US stocks minus the 1-month US Treasury bill rate
- **Size (SMB):** Returns of stocks categorized as **Small** (by market cap) **Minus** those of **Big equities**

- **Value (HML)**: Returns of stocks with **High** book-to-market value **Minus** those with a **Low value**
- **Investment (CMA)**: Returns differences for companies with **Conservative** investment expenditures **Minus** those with **Aggressive spending**
- **Profitability (RMW)**: Similarly, return differences for stocks with **Robust** profitability **Minus** that with a **Weak** metric.

We source the data from Kenneth French's data library using `pandas_datareader` (see *Chapter 4, Financial Feature Engineering – How to Research Alpha Factors*):

```
import pandas_datareader.data as web
factor_data = (web.DataReader('F-F_Research_Data_5_Factors_2x3_daily',
                              'famafrench', start=START)[0])
```

Next, we apply `statsmodels`' `RollingOLS()` to run regressions over windowed periods of different lengths, ranging from 15 to 90 days. We set the `params_only` parameter on the `.fit()` method to speed up computation and capture the coefficients using the `.params` attribute of the fitted `factor_model`:

```
factors = ['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA']
windows = list(range(15, 90, 5))
for window in windows:
    betas = []
    for symbol, data in universe.groupby(level='symbol'):
        model_data = data[['ret']].merge(factor_data, on='date').dropna()
        model_data[ret] -= model_data.RF
        rolling_ols = RollingOLS(endog=model_data[ret],
                                exog=sm.add_constant(model_data[factors]),
                                window=window)
        factor_model = rolling_ols.fit(params_only=True).params.drop('const',
                                                                    axis=1)
        result = factor_model.assign(symbol=symbol).set_index('symbol',
                                                            append=True)
        betas.append(result)
    betas = pd.concat(betas).rename(columns=lambda x: f'{window:02}_{x}')
    universe = universe.join(betas)
```

Features selecting based on mutual information

The next step is to select the 15 most relevant features from the 20 candidates to fill the 15×15 input grid. The code examples for the following steps are in the notebook `convert_cnn_features_to_image_format`.

To this end, we estimate the mutual information for each indicator and the 15 intervals with respect to our target, the one-day forward returns. As discussed in *Chapter 4, Financial Feature Engineering – How to Research Alpha Factors*, scikit-learn provides the `mutual_info_regression()` function that makes this straightforward, albeit time-consuming and memory-intensive. To accelerate the process, we randomly sample 100,000 observations:

```
df = features.join(targets[target]).dropna().sample(n=100000)
X = df.drop(target, axis=1)
y = df[target]
mi[t] = pd.Series(mutual_info_regression(X=X, y=y), index=X.columns)
```

The left panel in *Figure 18.16* shows the mutual information, averaged across the 15 intervals for each indicator. NATR, PPO, and Bollinger Bands are most important from this metric's perspective:

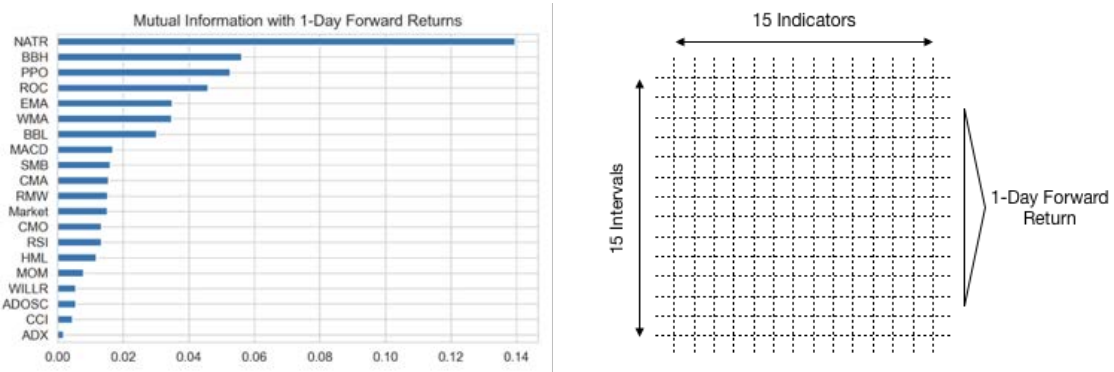


Figure 18.17: Mutual information and two-dimensional grid layout for time series

Hierarchical feature clustering

The right panel in *Figure 18.16* sketches the 15 X 15 two-dimensional feature grid that we will feed into our CNN. As discussed in the first section of this chapter, CNNs rely on the locality of relevant patterns that is typically found in images where nearby pixels are closely related and changes from one pixel to the next are often gradual.

To organize our indicators in a similar fashion, we will follow Sezer and Ozbayoglu's approach of applying hierarchical clustering. The goal is to identify features that behave similarly and order the columns and the rows of the grid accordingly.

We can build on SciPy's `pairwise_distance()`, `linkage()`, and `dendrogram()` functions that we introduced in *Chapter 13, Data-Driven Risk Factors and Asset Allocation with Unsupervised Learning* alongside other forms of clustering. We create a helper function that standardizes the input column-wise to avoid distorting distances among features due to differences in scale, and use the Ward criterion that merges clusters to minimize variance. The function returns the order of the leaf nodes in the dendrogram that in turn displays the successive formation of larger clusters:

```
def cluster_features(data, labels, ax, title):
    data = StandardScaler().fit_transform(data)
    pairwise_distance = pdist(data)
    Z = linkage(data, 'ward')
    dend = dendrogram(Z,
                      labels=labels,
                      orientation='top',
                      leaf_rotation=0.,
                      leaf_font_size=8.,
                      ax=ax)

    return dend['ivl']
```

To obtain the optimized order of technical indicators in the columns and the different intervals in the rows, we use NumPy's `.reshape()` method to ensure that the dimension we would like to cluster appears in the columns of the two-dimensional array we pass to `cluster_features()`:

```
labels = sorted(best_features)
col_order = cluster_features(features.dropna().values.reshape(-1, 15).T,
                             labels)

labels = list(range(1, 16))
row_order = cluster_features(
    features.dropna().values.reshape(-1, 15, 15).transpose((0, 2, 1)).
    reshape(-1, 15).T, labels)
```

Figure 18.18 shows the dendrograms for both the row and column features:

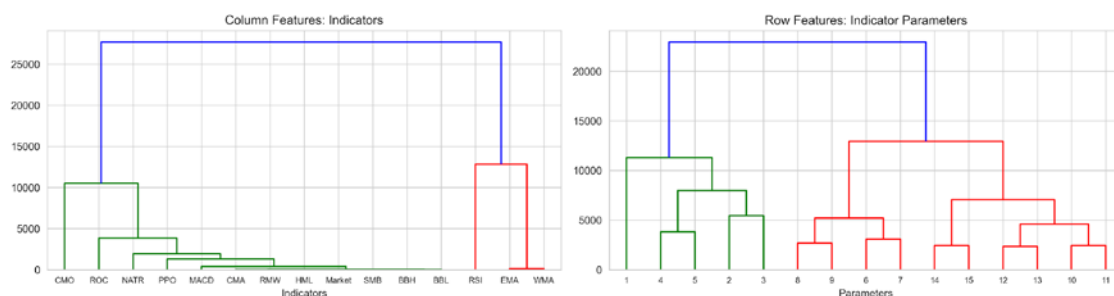


Figure 18.18: Dendrograms for row and column features

We reorder the features accordingly and store the result as inputs for the CNN that we will create in the next step.

Creating and training a convolutional neural network

Now we are ready to design, train, and evaluate a CNN following the steps outlined in the previous section. The notebook `cnn_for_trading.ipynb` contains the relevant code examples.

We again closely follow the authors in creating a CNN with 2 convolutional layers with kernel size 3 and 16 and 32 filters, respectively, followed by a max pooling layer of size 2. We flatten the output of the last stack of filters and connect the resulting 1,568 outputs to a dense layer of size 32, applying 25 and 50 percent dropout probability to the incoming and outgoing connections to mitigate overfitting. The following table summarizes the CNN structure that contains 55,041 trainable parameters:

Layer (type)	Output Shape	Param #
CONV1 (Conv2D)	(None, 15, 15, 16)	160
CONV2 (Conv2D)	(None, 15, 15, 32)	4640
POOL1 (MaxPooling2D)	(None, 7, 7, 32)	0
DROP1 (Dropout)	(None, 7, 7, 32)	0
FLAT1 (Flatten)	(None, 1568)	0
FC1 (Dense)	(None, 32)	50208
DROP2 (Dropout)	(None, 32)	0
FC2 (Dense)	(None, 1)	33
Total params:	55,041	
Trainable params:	55,041	
Non-trainable params:	0	

We cross-validate the model with the `MutipleTimeSeriesCV` train and validation set index generator introduced in *Chapter 7, Linear Models – From Risk Factors to Return Forecasts*. We provide 5 years of trading days during the training period in batches of 64 random samples and validate using the subsequent 3 months, covering the years 2014-2017.

We scale the features to the range $[-1, 1]$ and again use NumPy's `.reshape()` method to create the requisite $N \times 15 \times 15 \times 1$ format:

```
def get_train_valid_data(X, y, train_idx, test_idx):
    x_train, y_train = X.iloc[train_idx, :], y.iloc[train_idx]
    x_val, y_val = X.iloc[test_idx, :], y.iloc[test_idx]
    scaler = MinMaxScaler(feature_range=(-1, 1))
    x_train = scaler.fit_transform(x_train)
    x_val = scaler.transform(x_val)
    return (x_train.reshape(-1, size, size, 1), y_train,
            x_val.reshape(-1, size, size, 1), y_val)
```

Training and validation follow the process laid out in *Chapter 17, Deep Learning for Trading*, relying on checkpointing to store weights after each epoch and generate predictions for the best-performing iterations without the need for costly retraining.

To evaluate the model's predictive accuracy, we compute the daily **information coefficient (IC)** for the validation set like so:

```
checkpoint_path = Path('models', 'cnn_ts')
for fold, (train_idx, test_idx) in enumerate(cv.split(features)):
    X_train, y_train, X_val, y_val = get_train_valid_data(features, target,
                                                            train_idx, test_idx)
```

```

preds = y_val.to_frame('actual')
r = pd.DataFrame(index=y_val.index.unique(level='date')).sort_index()
model = make_model(filter1=16, act1='relu', filter2=32,
                    act2='relu', do1=.25, do2=.5, dense=32)
for epoch in range(n_epochs):
    model.fit(X_train, y_train,
              batch_size=batch_size,
              validation_data=(X_val, y_val),
              epochs=1, verbose=0, shuffle=True)
    model.save_weights(
        (checkpoint_path / f'ckpt_{fold}_{epoch}').as_posix())
    preds[epoch] = model.predict(X_val).squeeze()
    r[epoch] = preds.groupby(level='date').apply(
        lambda x: spearmanr(x.actual, x[epoch])[0]).to_frame(epoch)

```

We train the model for up to 10 epochs using **stochastic gradient descent** with **Nesterov** momentum (see *Chapter 17, Deep Learning for Trading*) and find that the best performing epochs, 8 and 9, achieve a (low) daily average IC of around 0.009.

Assembling the best models to generate tradeable signals

To reduce the variance of the test-period forecasts, we generate and average the predictions for the 3 models that perform best during cross-validation, which here correspond to training for 4, 8, and 9 epochs. As in the previous time-series example, the relatively short training period underscores that the amount of signals in financial time series is low compared to the systematic information contained in, for example, image data.

The `generate_predictions()` function reloads the model weights and returns the forecasts for the target period:

```

def generate_predictions(epoch):
    predictions = []
    for fold, (train_idx, test_idx) in enumerate(cv.split(features)):
        X_train, y_train, X_val, y_val = get_train_valid_data(
            features, target, train_idx, test_idx)
        preds = y_val.to_frame('actual')
        model = make_model(filter1=16, act1='relu', filter2=32,
                            act2='relu', do1=.25, do2=.5, dense=32)
        status = model.load_weights(
            (checkpoint_path / f'ckpt_{fold}_{epoch}').as_posix())
        status.expect_partial()
        predictions.append(pd.Series(model.predict(X_val).squeeze(),
                                      index=y_val.index))

    return pd.concat(predictions)

preds = {}
for i, epoch in enumerate(ic.drop('fold', axis=1).mean().nlargest(3).index):
    preds[i] = generate_predictions(epoch)

```

We store the predictions and proceed to backtest a trading strategy based on these daily return forecasts.

Backtesting a long-short trading strategy

To get a sense of the signal quality, we compute the spread between equally weighted portfolios invested in stocks selected according to the signal quintiles using Alphalens (see *Chapter 4, Financial Feature Engineering – How to Research Alpha Factors*).

Figure 18.19 shows that for a one-day investment horizon, this naive strategy would have earned a bit over four basis points per day during the 2013-2017 period:

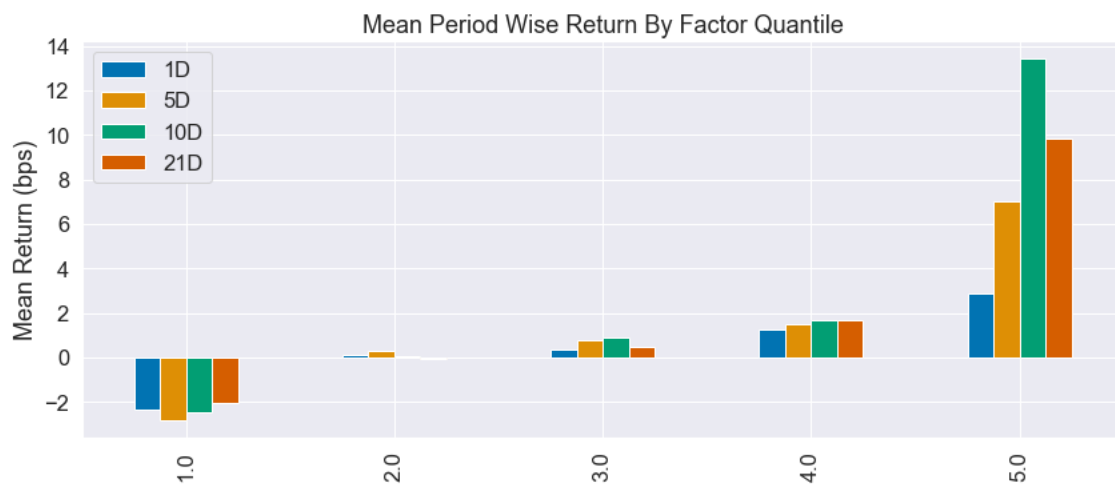


Figure 18.19: Alphalens signal quality evaluation

We translate this slightly encouraging result into a simple strategy that enters long (short) positions for the 25 stocks with the highest (lowest) return forecasts, trading on a daily basis. Figure 18.20 shows that this strategy is competitive with the S&P 500 benchmark over much of the backtesting period (left panel), resulting in a 35.6 percent cumulative return and a Sharpe ratio of 0.53 (before transaction costs; right panel)

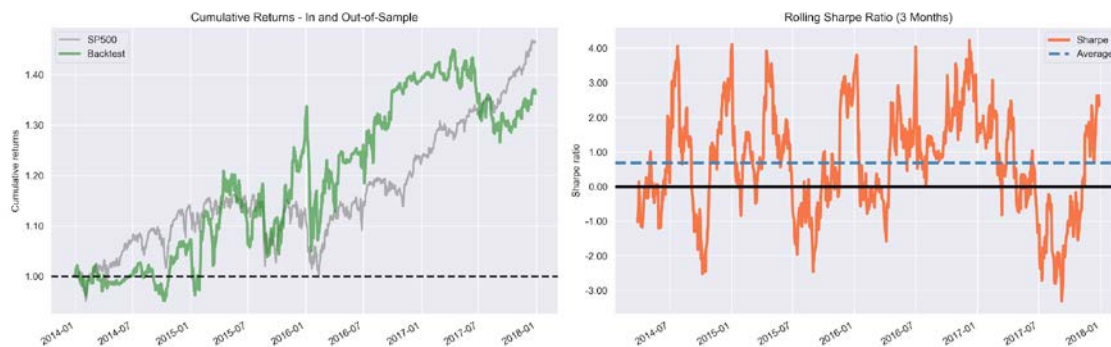


Figure 18.20: Backtest performance in- and out-of-sample

Summary and lessons learned

It appears that the CNN is able to extract meaningful information from the time series of alpha factors converted into a two-dimensional grid. Experimentation with different architectures and training parameters shows that the result is not very robust and slight modifications can yield significantly worse performance.

Tuning attempts also surface the notorious difficulties in successfully training a deep NN, especially when the signal-to-noise ratio is low: too complex a network or the wrong optimizer can lead the CNN to a local optimum where it always predicts a constant value.

The most important step to improve the results and obtain a performance closer to that achieved by the authors (using different outcomes) would be to revisit the features. There are many alternatives to different intervals of a limited set of technical indicators. Any appropriate number of time-series features could be arranged in a rectangular $n \times m$ format and benefit from the CNN's ability to learn local patterns. The choice of n indicators and m intervals just makes it easier to organize the rows and the columns of the two-dimensional grid. Give it a shot!

Furthermore, the authors take a classification approach to the algorithmically labeled buy, hold, and sell outcomes (see the paper for an outline of the computation), whereas our experiment applied regression to the daily returns. The Alphalens chart in *Figure 18.18* suggests that longer holding periods (especially 10 days) might work better, so there is also scope for adjusting the strategy accordingly or switching to a classification approach.

Summary

In this chapter, we introduced CNNs, a specialized NN architecture that has taken cues from our (limited) understanding of human vision and performs particularly well on grid-like data. We covered the central operation of convolution or cross-correlation that drives the discovery of filters that in turn detect features useful to solve the task at hand.

We reviewed several state-of-the-art architectures that are good starting points, especially because transfer learning enables us to reuse pretrained weights and reduce the otherwise rather computationally and data-intensive training effort. We also saw that Keras makes it relatively straightforward to implement and train a diverse set of deep CNN architectures.

In the next chapter, we turn our attention to recurrent neural networks that are designed specifically for sequential data, such as time-series data, which is central to investment and trading.

